

Qt OpenGL Scrolling Game Window Template - 53D

Reusable starter template for a story/tutorial game window with OpenGL drawing, world-space positions, camera scrolling, parallax background movement, and platform tiles.

Important Ideas

World Space vs Screen Space: Store gameplay positions in world coordinates, then draw them with camera subtraction. `Player_World_X` is where the player really is in the level. `Player_World_X - Camera_X` is where the player appears on the screen.

Camera Follow: `Update_Camera` keeps the player around 35% from the left side of the screen. Clamp `Camera_X` to 0 so the camera cannot scroll before the level starts.

Non-Repeating Parallax Background: For one long background image, do not use modulo (%) and do not draw a second repeated copy. Draw the long image once at `-(Camera_X * Parallax_Speed)`.

Platforms Now, Tiles Later: `Platform_Tile` uses X, Y, W, H now, but `Tile_Type` lets you later choose left/middle/right tile images from a tileset.

Collision: The starter collision checks whether the player is falling, horizontally overlapping a platform, and crossing the platform top. This is intentionally simple and can be expanded later.

OpenGL and QPainter: This template uses QPainter for images and OpenGL immediate-mode rectangles for temporary platform drawing. Later, platform rectangles can be replaced with QPainter tile images or OpenGL textured quads.

Header Template

```
#ifndef SCROLLING_GAME_WINDOW_H
#define SCROLLING_GAME_WINDOW_H

#include <QOpenGLWidget>
#include <QOpenGLFunctions>
#include <QTimer>
#include <QKeyEvent>
#include <QImage>
#include <QList>

class Scrolling_Game_Window : public QOpenGLWidget, protected QOpenGLFunctions
{
    Q_OBJECT

public:
    explicit Scrolling_Game_Window(QWidget *parent = nullptr);

protected:
    void initializeGL() override;
    void resizeGL(int w, int h) override;
    void paintGL() override;
    void keyPressEvent(QKeyEvent *event) override;
    void keyReleaseEvent(QKeyEvent *event) override;

private:
    struct Platform_Tile
    {
        float X;
        float Y;
        float W;
        float H;
        int Tile_Type; // 0 = normal, later: left/middle/right/edge tiles
    };

    QTimer *Game_Timer;

    QImage Background_Image;
    QImage Player_Sheet;
    QImage Tile_Set_Image; // optional later when platforms use tiles

    QList<Platform_Tile> Platform_Tiles;

    float Camera_X;
    float Camera_Y;
    float Parallax_Speed; // 0.4 means background moves at 40% camera speed

    float Player_World_X;
    float Player_World_Y;
    float Player_Vx;
    float Player_Vy;
};
```

```
float Gravity;
bool Player_Grounded;

bool Move_Left_Pressed;
bool Move_Right_Pressed;
bool Move_Jump_Pressed;

int Player_Frame;
int Player_Frame_Count;
int Player_Frame_W;
int Player_Frame_H;

int Get_Player_Draw_Size() const;
void Create_Level();
void Update_Camera();
void Draw_Background(QPainter &painter);
void Draw_Platforms();
void Draw_Rect(float x, float y, float w, float h);
void Handle_Platform_Collision(int playerDrawSize);

private slots:
    void Update_Game();
};

#endif // SCROLLING_GAME_WINDOW_H
```

CPP Template

```
#include "scrolling_game_window.h"

#include <QPainter>
#include <QRect>
#include <QDebug>

static const char *BACKGROUND_PATH = ":/Images/Background/Long_Background.png";
static const char *PLAYER_SHEET_PATH = ":/Images/Characters/Walking_Gray.png";

Scrolling_Game_Window::Scrolling_Game_Window(QWidget *parent)
    : QOpenGLWidget(parent)
{
    Game_Timer = new QTimer(this);

    Background_Image = QImage(BACKGROUND_PATH);
    Player_Sheet = QImage(PLAYER_SHEET_PATH);

    Camera_X = 0;
    Camera_Y = 0;
    Parallax_Speed = 0.4f;

    Player_World_X = 150;
    Player_World_Y = 350;
    Player_Vx = 0;
    Player_Vy = 0;
    Gravity = 1200.0f;
    Player_Grounded = false;

    Move_Left_Pressed = false;
    Move_Right_Pressed = false;
    Move_Jump_Pressed = false;

    Player_Frame = 0;
    Player_Frame_Count = 8;
    Player_Frame_W = 32;
    Player_Frame_H = 32;

    Create_Level();

    connect(Game_Timer, SIGNAL(timeout()), this, SLOT(Update_Game()));
    Game_Timer->start(17); // roughly 60 fps

    setFocusPolicy(Qt::StrongFocus);
}

void Scrolling_Game_Window::Create_Level()
{
    Platform_Tiles.append({100, 500, 300, 35, 0});
    Platform_Tiles.append({550, 420, 300, 35, 0});
    Platform_Tiles.append({1000, 500, 300, 35, 0});
    Platform_Tiles.append({1500, 460, 300, 35, 0});
}

void Scrolling_Game_Window::initializeGL()
{
    initializeOpenGLFunctions();
    glClearColor(0.02f, 0.02f, 0.08f, 1.0f);
}

void Scrolling_Game_Window::resizeGL(int w, int h)
{
    glViewport(0, 0, w, h);
}

void Scrolling_Game_Window::paintGL()
{
    glClear(GL_COLOR_BUFFER_BIT);

    QPainter painter(this);
    Draw_Background(painter);

    painter.beginNativePainting();
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0, width(), height(), 0, -1, 1);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glColor3f(0.2f, 0.2f, 0.2f);
    Draw_Platforms();
    painter.endNativePainting();

    int sourceX = Player_Frame * Player_Frame_W;
    QRect sourceRect(sourceX, 0, Player_Frame_W, Player_Frame_H);

    int playerDrawSize = Get_Player_Draw_Size();
    QRect targetRect(Player_World_X - Camera_X,
                     Player_World_Y - Camera_Y,
                     playerDrawSize,
```

```

        playerDrawSize);

    painter.drawImage(targetRect, Player_Sheet, sourceRect);
    painter.end();
}

void Scrolling_Game_Window::Draw_Background(QPainter &painter)
{
    if(Background_Image.isNull())
        return;

    float scale = (float)height() / Background_Image.height();
    int scaledBackgroundWidth = Background_Image.width() * scale;

    int backgroundX = -(Camera_X * Parallax_Speed);

    painter.drawImage(QRect(backgroundX, 0, scaledBackgroundWidth, height()),
        Background_Image);
}

void Scrolling_Game_Window::Draw_Platforms()
{
    for(int i = 0; i < Platform_Tiles.size(); i++)
    {
        Platform_Tile tile = Platform_Tiles[i];

        Draw_Rect(tile.X - Camera_X,
            tile.Y - Camera_Y,
            tile.W,
            tile.H);
    }
}

void Scrolling_Game_Window::Update_Game()
{
    float dt = 0.017f;
    int playerDrawSize = Get_Player_Draw_Size();

    if(Move_Left_Pressed)
        Player_Vx = -500;
    else if(Move_Right_Pressed)
        Player_Vx = 500;
    else
        Player_Vx = 0;

    if(Move_Jump_Pressed && Player_Grounded)
    {
        Player_Vy = -700;
        Player_Grounded = false;
    }

    Player_Vy += Gravity * dt;
    Player_World_X += Player_Vx * dt;
    Player_World_Y += Player_Vy * dt;

    Handle_Platform_Collision(playerDrawSize);
    Update_Camera();

    Player_Frame++;
    if(Player_Frame >= Player_Frame_Count)
        Player_Frame = 0;

    update();
}

void Scrolling_Game_Window::Update_Camera()
{
    Camera_X = Player_World_X - width() * 0.35f;

    if(Camera_X < 0)
        Camera_X = 0;

    Camera_Y = 0; // keep vertical camera fixed until vertical scrolling is needed
}

void Scrolling_Game_Window::Handle_Platform_Collision(int playerDrawSize)
{
    Player_Grounded = false;

    for(int i = 0; i < Platform_Tiles.size(); i++)
    {
        Platform_Tile tile = Platform_Tiles[i];

        bool falling = Player_Vy >= 0;
        bool overlapsX = Player_World_X + playerDrawSize > tile.X &&
            Player_World_X < tile.X + tile.W;
        bool crossesTop = Player_World_Y + playerDrawSize >= tile.Y &&
            Player_World_Y + playerDrawSize <= tile.Y + tile.H;

        if(falling && overlapsX && crossesTop)
        {
            Player_World_Y = tile.Y - playerDrawSize;

```

```

        Player_Vy = 0;
        Player_Grounded = true;
        return;
    }
}

void Scrolling_Game_Window::Draw_Rect(float x, float y, float w, float h)
{
    glBegin(GL_QUADS);
    glVertex2f(x, y);
    glVertex2f(x + w, y);
    glVertex2f(x + w, y + h);
    glVertex2f(x, y + h);
    glEnd();
}

int Scrolling_Game_Window::Get_Player_Draw_Size() const
{
    return qMax(32, (96 * qMax(width(), 320)) / 1280);
}

void Scrolling_Game_Window::keyPressEvent(QKeyEvent *event)
{
    if(event->key() == Qt::Key_A || event->key() == Qt::Key_Left)
        Move_Left_Pressed = true;
    else if(event->key() == Qt::Key_D || event->key() == Qt::Key_Right)
        Move_Right_Pressed = true;
    else if(event->key() == Qt::Key_Space)
        Move_Jump_Pressed = true;

    QOpenGLWidget::keyPressEvent(event);
}

void Scrolling_Game_Window::keyReleaseEvent(QKeyEvent *event)
{
    if(event->key() == Qt::Key_A || event->key() == Qt::Key_Left)
        Move_Left_Pressed = false;
    else if(event->key() == Qt::Key_D || event->key() == Qt::Key_Right)
        Move_Right_Pressed = false;
    else if(event->key() == Qt::Key_Space)
        Move_Jump_Pressed = false;

    QOpenGLWidget::keyReleaseEvent(event);
}

```

Checklist For Using This Template

- Add the new .h and .cpp files to the .pro SOURCES and HEADERS lists.
- Replace BACKGROUND_PATH and PLAYER_SHEET_PATH with exact qrc paths from Resources.qrc.
- Call setFocusPolicy(Qt::StrongFocus) so key events reach the game widget.
- Keep gameplay movement in world coordinates, not raw screen coordinates.
- Draw world objects using world position minus camera position.
- For a long non-repeating background, clamp the camera before the background image runs out.
- Replace Draw_Rect platform rendering with tile image drawing when tileset art is ready.